

Guia Prático — Frontend

Projeto ArenaPlay (trabalhoprogweb) · Como configurar estilos, layouts e campos da tela

1. O projeto tem DOIS sistemas de estilo

Entender isto evita 90% da confusão. Cada tipo de página usa um sistema diferente:

Sistema	Onde é usado	Como o estilo é aplicado
Bootstrap 5 + CSS próprio	Páginas públicas e painéis (welcome, arenas, dashboards) que usam <code>@extends('layouts.main')</code>	Classes do Bootstrap (<code>container</code> , <code>btn</code> , <code>card</code> ...) + classes próprias em <code>public/css/style.css</code>
Tailwind + componentes Jetstream	Telas que vieram prontas do Jetstream e nunca foram reescritas: recuperar senha, redefinir senha, verificar e-mail, confirmar senha, autenticação em duas etapas, termos de uso, política de privacidade e o perfil do Jetstream. Usam <code><x-guest-layout></code> ou <code><x-app-layout></code> .	Classes do Tailwind (<code>mt-4</code> , <code>w-full</code> ...) direto no HTML + componentes <code><x-input></code> , <code><x-button></code>

Regra de ouro: olhe a primeira linha da view. Se começa com `@extends('layouts.main')` → é **Bootstrap**. Se começa com `<x-guest-layout>` ou `<x-app-layout>` → é **Tailwind**.

Isto mudou. As telas de **login, cadastro de cliente** (`register`) e **cadastro de arena** (`registerArenaOwners`) eram Tailwind e **hoje são Bootstrap** (`@extends('layouts.main')`). O projeto está migrando para Bootstrap: ao criar uma tela nova, use **Bootstrap**, mesmo sendo de autenticação.

Por que o Tailwind ainda não pode ser removido: as telas da linha acima dependem dele, e o Tailwind é compilado pelo **Vite**. Por isso o `npm run build` continua obrigatório na instalação — sem ele, essas telas quebram com `"Vite manifest not found"`.

2. Mapa dos arquivos de frontend

Arquivo	O que controla
<code>resources/views/layouts/main.blade.php</code>	Layout base das páginas públicas (navbar, <head>, onde entra o conteúdo)
<code>public/css/style.css</code>	Seus estilos customizados (cores, cards, botões, navbar)
<code>public/js/script.js</code>	JavaScript próprio das páginas públicas
<code>public/js/masks.js</code>	Máscaras de digitação: telefone, CPF/CNPJ, moeda e percentual
<code>public/js/form-wizard.js</code>	Formulário dividido em etapas (usado no cadastro de arena)
<code>public/css/business-hours.css</code> · <code>courts.css</code> · <code>payment-methods.css</code>	Estilo dos blocos reutilizáveis de horários, quadras e formas de pagamento
<code>resources/views/components/</code>	Componentes Blade reutilizáveis (<code>x-input</code> , <code>x-button</code> , <code>x-label</code> ...) — usados apenas pelas telas Tailwind restantes
<code>tailwind.config.js</code>	Configuração do Tailwind (fontes, cores do tema)
<code>resources/views/<pasta>/*.blade.php</code>	As telas em si (o HTML de cada página)

3. Mudar estilos das páginas públicas (Bootstrap)

O CSS próprio fica em `public/css/style.css` e já é carregado pelo layout. Para mudar cores e aparência, edite as classes que já existem lá. Principais classes do seu projeto:

Classe	O que estiliza
<code>.custom-navbar</code>	Fundo da barra de navegação (gradiente azul-escuro)
<code>.logo</code>	Texto "ArenaPlay" do topo
<code>.nav-link</code> / <code>.nav-link:hover</code>	Links do menu (cor normal e ao passar o mouse)
<code>.btn-register</code>	Botão amarelo "CADASTRAR"
<code>.dashboard-card</code>	Cartões dos painéis
<code>.dashboard-btn-primary</code>	Botão principal dos painéis
<code>.logo-marca</code>	"ArenaPlay" em fundo claro (cards de login e cadastro). A <code>.logo</code> é branca e só serve na navbar escura.
<code>.wz-passos</code> / <code>.wz-passo</code>	Indicador de etapas do cadastro de arena (1 a 4)
<code>.slide-legenda</code>	Título e subtítulo sobre a foto do carrossel da tela inicial
<code>.min-w-0</code>	Utilitário que o Bootstrap não tem: libera um item de flex/grid a encolher. Sem ele, textos longos estouram o container em telas estreitas.
<code>.auth-box</code> / <code>.auth-container</code> / <code>.auth-card</code> / <code>.auth-title</code> / <code>.auth-input</code> / <code>.auth-btn</code>	CSS morto. Existem em <code>style.css</code> , mas nenhuma view as usa desde que login e cadastro passaram a usar o card padrão do Bootstrap. Podem ser removidas.

Paleta de cores do projeto

Cor	Código	Onde aparece
 Azul-escuro	<code>#021B35</code>	Navbar, botões principais
 Azul-noite	<code>#001427</code>	Final do gradiente da navbar
 Amarelo	<code>#f6c026</code>	Botão CADASTRAR, hover dos links
 Cinza-claro	<code>#f3f4f6</code>	Fundo do site (body)

Exemplo: mudar a cor do botão "CADASTRAR"

```
/* public/css/style.css */
.btn-register{
  background-color: #16a34a; /* trocar de amarelo para verde */
  color: #fff;
  padding: 14px 30px;
  border-radius: 12px;
}
```

Dica: arquivos em `public/` são servidos diretamente. Basta salvar o `style.css` e dar **F5** no navegador (Ctrl+F5 se não atualizar — limpa o cache).

Classes utilitárias do Bootstrap mais usadas

Classe	O que faz
<code>container</code> / <code>container-fluid</code>	Centraliza e limita a largura do conteúdo
<code>row</code> + <code>col-md-4</code>	Grade: linha com colunas (4 de 12 = 1/3 da largura)
<code>btn btn-primary</code>	Botão azul padrão (variações: <code>btn-success</code> , <code>btn-danger</code> , <code>btn-outline-primary</code>)
<code>card</code> / <code>card-body</code>	Cartão com borda e sombra
<code>mb-3</code> , <code>mt-4</code> , <code>p-4</code> , <code>py-5</code>	Espaçamentos: m=margem, p=padding; b/t/x/y = baixo/topo/lados; 0-5 = intensidade
<code>d-flex</code> + <code>justify-content-end</code> + <code>gap-2</code>	Alinhamento flexível (itens lado a lado, à direita, com espaço)
<code>form-control</code>	Estilo padrão de input/textarea
<code>shadow-sm</code> / <code>shadow</code>	Sombra suave / média

4. Como funciona o LAYOUT base (Bootstrap)

O layout é o "molde" da página. Em `layouts/main.blade.php` existe a navbar fixa e um ponto onde o conteúdo de cada página é injetado:

```
<!-- layouts/main.blade.php (resumido) -->
<body>
  <nav> ... menu ... </nav>
  <main>
    @yield('content') <!-- AQUI entra o conteudo de cada pagina -->
  </main>
</body>
```

Cada página "preenche" esse molde:

```
<!-- resources/views/Arenas/index.blade.php -->
@extends('layouts.main')      {{-- usa o molde --}}
@section('title', 'Minhas Arenas') {{-- preenche o @yield('title') --}}

@section('content')            {{-- preenche o @yield('content') --}}
  <h1>Minhas Arenas</h1>
@endsection
```

No layout	Na página	Função
<code>@yield('content')</code>	<code>@section('content') ... @endsection</code>	Bloco grande de conteúdo
<code>@yield('title')</code>	<code>@section('title', 'Texto')</code>	Valor curto (título da aba)

Para mudar a navbar de todas as páginas de uma vez, edite `layouts/main.blade.php` — a alteração reflete em toda página que usa `@extends('layouts.main')`.

5. Componentes Blade (telas do Jetstream — Tailwind)

Não use isto em telas novas. Estes componentes servem apenas às telas que ainda são Tailwind (recuperar senha, verificar e-mail, termos, política...). O **login e os cadastros já não os usam** — eles são Bootstrap e escrevem os campos direto no HTML, com `<label>` e `class="form-control"`.

Essas telas usam **componentes reutilizáveis** (começam com `x-`). Eles ficam em `resources/views/components/`. Em vez de escrever um `<input>` com 10 classes, você escreve:

```
<x-label for="email" value="Email" />
<x-input id="email" type="email" name="email" class="block mt-1 w-full" required />
<x-button>Salvar</x-button>
```

Componente	Vira	Arquivo
<code><x-input /></code>	Um <code><input></code> estilizado	components/input.blade.php
<code><x-label /></code>	Uma <code><label></code>	components/label.blade.php
<code><x-button>...</x-button></code>	Um <code><button></code> escuro	components/button.blade.php
<code><x-validation-errors /></code>	Lista de erros de validação	components/validation-errors.blade.php

Customizar a aparência de um componente

Para mudar o estilo de TODOS os inputs de auth de uma vez, edite o componente. Ex.: `components/input.blade.php` :

```
<input {!! $attributes->merge([
    'class' => 'border-gray-300 focus:border-indigo-500 rounded-md shadow-sm'
]) !!}>
```

Troque as classes Tailwind dentro de `'class' => '...'` . O `$attributes->merge` junta essas classes-padrão com qualquer `class="..."` que você passar na hora de usar o componente.

Tailwind x Bootstrap: no Tailwind, o estilo é montado por muitas classes pequenas no próprio HTML (`px-4 py-2 bg-gray-800 rounded-md`). No Bootstrap, você usa classes prontas maiores (`btn btn-primary`). Não misture os dois na mesma tela.

6. RECEITA: adicionar um novo campo a um formulário

Adicionar um campo na tela exige tocar em até 4 lugares (de cima pra baixo no fluxo). Exemplo: adicionar o campo **"site"** numa arena.

1. **View (o campo na tela)** — adicione o input no formulário:

```
<div class="mb-3">
    <input type="text" name="site" class="form-control"
        placeholder="Site da arena">
</div>
```

2. **Controller (validar + salvar)** — inclua o campo na validação e no create:

```
$validated = $request->validate([
    // ... campos existentes ...
    'site' => ['nullable', 'url', 'max:255'],
]);
Arena::create([ /* ... */ 'website' => $validated['site'] ?? null ]);
```

3. **Model (permitir salvar)** — adicione a coluna ao `$fillable` :

```
protected $fillable = [ /* ... */ 'website' ];
```

4. **Migration (criar a coluna no banco)** — crie e rode uma migration:

```
php artisan make:migration add_website_to_arenas_table

// dentro do up():
$table->string('website', 255)->nullable();

php artisan migrate
```

Cuidado com os nomes! O `name="site"` do input é o que o controller lê em `$request`. A coluna no banco (`website`) é o que vai no `$fillable` e no `create`. Eles podem ter nomes diferentes — só não confunda qual usar em cada lugar.

7. RECEITA: exibir dados na tela

Para mostrar dados que o controller enviou (ex.: a lista de arenas em `$arenas`):

```
@foreach($arenas as $arena)
    <div class="card mb-3">
        <div class="card-body">
            <h5>{{ $arena->name }}</h5>
            <p>{{ $arena->address }}</p>
        </div>
    </div>
@endforeach
```

Quero...	Uso
Imprimir um valor	<code>{{ \$arena->name }}</code>
Repetir para cada item	<code>@foreach(\$lista as \$item) ... @endforeach</code>
Mostrar só se houver dados	<code>@if(\$arenas->count()) ... @else Nenhuma arena @endif</code>
Valor padrão se vazio	<code>{{ \$arena->phone ?? 'Sem telefone' }}</code>
Link para outra página	<code>id) }}"></code>

Lembre-se: as colunas no banco são em inglês (`name`, `address`, `phone`). Usar `$arena->nome` mostra **vazio**. Use sempre o nome real da coluna.

8. Checklist rápido de frontend

- Tela nova (qualquer uma, inclusive de autenticação)?** Use `@extends('layouts.main')` + classes Bootstrap. Não crie telas novas em Tailwind.
- Vai **mexer** numa tela Tailwind que já existe (recuperar senha, termos...)? Aí sim use `<x-guest-layout>` + componentes `x-`.
- Campo de telefone, CPF/CNPJ, dinheiro ou porcentagem? Adicione `data-mask="telefone"` (ou `cpfcpnpj`, `moeda`, `percentual`) e carregue `/js/masks.js`.
- Algo estourando a largura no celular? Quase sempre é item de flex/grid sem `min-width: 0`. Use a classe `.min-w-0`.
- Quer mudar cor/estilo global das páginas públicas? Edite `public/css/style.css`.

6. Quer mudar a navbar de todas as páginas? Edite `layouts/main.blade.php`.
7. Novo campo no form? View → Controller (validar) → Model (`$fillable`) → Migration.
8. Todo formulário POST precisa de `@csrf` logo após a tag `<form>`.
9. Para exibir dados, use o nome **real** da coluna (inglês), não o do formulário.